Kadane's Algorithm: The 3-Step LeetCode Progression

To handle the **"Medium: MSS With Swaps"** problem from the Infosys 2026 sample, you need to be comfortable using Kadane's algorithm when the rules of the array change slightly.

Do these three free LeetCode problems in order today.

Level 1: The Warm-Up

LeetCode 53: Maximum Subarray

- **The Goal:** Find the contiguous subarray with the largest sum.
- **Why do it:** This is the pure, vanilla Kadane's algorithm. You need to be able to type this out from memory in under 3 minutes without looking at any notes.
- **Infosys Connection:** This proves your base `current_sum` and `max_sum` logic is flawless.

Python Solution Template

```
class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        current_sum = max_sum = nums[0]

        for i in range(1, len(nums)):
            # Start a new streak or add to the current one?
            current_sum = max(nums[i], current_sum + nums[i])
            max_sum = max(max_sum, current_sum)

        return max_sum
```

Level 2: The Twist (Prefix/Suffix Tracking)

LeetCode 152: Maximum Product Subarray

- **The Goal:** Find a contiguous subarray that has the largest *product* (multiplication instead of addition).
- **Why do it:** When you multiply two negative numbers, they become a huge positive number. A single forward pass of Kadane's fails if there is an even number of negatives.
- **The Solution (Your Code!):** Run a forward and backward pass simultaneously. If you hit a `0`, reset the product for that specific direction.
- **Infosys Connection:** In the "MSS With Swaps" problem, you have to track multiple possibilities (e.g., "What if I swap this negative out?"). Learning to track two states (prefix and suffix) in one loop is the secret to solving the Medium tier.

Python Solution Template

```
class Solution:
    def maxProduct(self, nums: List[int]) -> int:
        prod = 1
        maxprod = nums[0]
        i = 0
```

```
        revprod = 1
        maxrevprod = nums[0]
        j = len(nums) - 1

        while i < len(nums):
            # Forward pass
            prod = prod * nums[i]
            maxprod = max(prod, maxprod)
            if nums[i] == 0:
                prod = 1

            # Backward pass
            revprod = revprod * nums[j]
            maxrevprod = max(revprod, maxrevprod)
            if nums[j] == 0:  # Reset based on the j pointer!
                revprod = 1

            i += 1
            j -= 1

        return max(maxprod, maxrevprod)
```

Level 3: The Infosys-Level Boss

LeetCode 918: Maximum Sum Circular Subarray

- **The Goal:** Find the maximum subarray sum, but the array is circular (the end connects to the beginning).
- **Why do it:** This is exactly the kind of "trick" Infosys pulls.
- **The Hint:** You solve this by running Kadane's twice. First, run standard Kadane's to find the normal max sum. Second, run Kadane's to find the *minimum* subarray sum, and subtract it from the total sum of the array. The answer is the maximum of those two results!
- **Infosys Connection:** This teaches you how to mathematically manipulate the result of Kadane's to bypass complex array manipulations (like simulating circular connections or, in your case, simulating K swaps).

Python Solution Template

```
class Solution:
    def maxSubarraySumCircular(self, nums: List[int]) -> int:
        total_sum = 0
        curr_max = curr_min = 0
        global_max = nums[0]
        global_min = nums[0]

        for num in nums:
            total_sum += num

            # Kadane's for Max
            curr_max = max(curr_max + num, num)
            global_max = max(global_max, curr_max)

            # Kadane's for Min
            curr_min = min(curr_min + num, num)
            global_min = min(global_min, curr_min)
```

```
    # Edge case: If all numbers are negative, global_max is the answer
    if global_max < 0:
        return global_max

    # The answer is either the normal max, or the circular max (total - min
    return max(global_max, total_sum - global_min)
```

Pro-Tip for "MSS With Swaps" (Infosys Medium)

Once you finish these three, you will realize that "MSS With Swaps" is just Kadane's algorithm where you are allowed to be Greedy. If your Kadane's streak is being ruined by a negative number, and you have a swap available ($K > 0$), you swap that negative number with the largest positive number available elsewhere in the array!